

End Term Project Report

Traffic Sign Detection and Classification using YOLOv5 and CNN

July 3, 2025

1. Introduction

Traffic sign recognition is a critical component of autonomous driving systems. This project combines the power of deep learning techniques, using YOLOv5 for object detection and CNN for image classification, to detect and classify traffic signs in natural scenes. The final model is integrated and deployed via a web application using Flask.

2. Objectives

- Detect traffic signs from images using YOLOv5.
- Classify detected signs using a CNN trained on the GTSRB dataset.
- Integrate both models into a working pipeline.
- Deploy the solution locally using Flask.

3. Dataset Description

3.1 Detection Dataset (GTSDDB)

- Dataset used: German Traffic Sign Detection Benchmark.
- Contains annotated images with various types of traffic signs.
- Labels were converted into YOLO format with normalized coordinates.
- **Classes:** 43 different traffic signs.
- YAML configuration:

```
train: /path/to/train/images
val: /path/to/val/images
nc: 43
names: ['0', '1', ..., '42']
```

3.2 Classification Dataset (GTSRB)

- Dataset used: German Traffic Sign Recognition Benchmark.
- Original images were grouped by class using Test.csv file.
- Directory structure organized for Keras `flow_from_directory()` usage.

4. YOLOv5 Model for Detection

4.1 Model Setup

- Pretrained model: `yolov5s.pt` (small version).
- Training framework: PyTorch using Ultralytics YOLO repository.
- Input image size: 640x640

4.2 Training Command

```
!python train.py --img 640 --batch 16 --epochs 10
--data data.yaml --weights yolov5s.pt --name gtsdb_detector
```

4.3 Results

- Mean Average Precision @ IoU=0.5 (mAP@0.5): 84–87%
- Detection of multiple traffic signs in complex scenes.
- Final weights saved to: `runs/detect/train/weights/best.pt`

5. CNN for Traffic Sign Classification

5.1 Preprocessing and Augmentation

- Images resized to 64x64 pixels.
- Data Augmentation:
 - Random rotations, zoom, width and height shifts.
 - Normalization using `rescale=1./255`.

5.2 Model Architecture

Conv2D(32, 3x3) + ReLU + MaxPool
Conv2D(64, 3x3) + ReLU + MaxPool
Conv2D(128, 3x3) + ReLU + MaxPool
Flatten + Dropout(0.5)
Dense(256) + ReLU
Dense(43) + Softmax

5.3 Training Details

- Optimizer: Adam
- Loss: Categorical Crossentropy
- Epochs: 30
- Batch size: 32
- Validation Accuracy: **92–94%**

6. Model Integration Pipeline

1. Image is passed to YOLO model for bounding box prediction.
2. Each region is cropped and resized to 64x64.
3. Cropped region is passed through the CNN model for classification.
4. The final output includes the image with labeled bounding boxes.

7. Flask Deployment

- Web interface developed using Flask.
- HTML/CSS used for frontend styling and file upload interface.
- Backend loads both models and processes uploaded images.
- Output image is returned with annotations.

8. Results and Visualization

- YOLO detection visualized with bounding boxes.
- CNN classification accuracy graph plotted per epoch.
- Final image overlayed with predicted labels.

9. Learnings

- Implemented object detection and image classification from scratch.
- Understood dataset formatting and data augmentation techniques.
- Integrated two models into a real-time pipeline.
- Gained experience in deploying deep learning models on the web.

10. Applications and Future Work

Applications

- Driver assistance systems in autonomous vehicles.
- Smart surveillance and traffic monitoring.
- Real-time road safety alert systems.

Future Scope

- Replace YOLOv5 with YOLOv8 for improved accuracy.
- Use webcam/video stream for real-time detection.
- Deploy app to a cloud platform like AWS or Heroku.
- Expand to include traffic light, pedestrian detection.